How the CRA tab calculates

A complete walkthrough of every formula, constant, and code path that produces the monthly PD7A remittance figures shown in NorthPay's CRA tab — derived directly from `lib/services/remittance.ts`, `lib/payroll/engine.ts`, and the supporting tax modules.

Document overview

The CRA tab does not run its own calculation engine. It is a **read-only aggregator** over the payroll runs the operator has already finalized. Every dollar on the screen is the sum of one specific field on each `PayrollLineResult`, bucketed by the calendar month of the `payDate`. To understand what the CRA tab shows, you need to understand two things:

- **How each line is computed** — federal tax, provincial tax, CPP1, CPP2, EI employee, EI employer. This is the work of the payroll engine and is summarized in Sections 3–7.
- **How those lines are folded** into monthly remittance buckets, due dates, and operator-marked status. This is the work of `RemittanceService` in Section 2.

All numeric constants in this document are the values defined in `lib/payroll/constants.ts` for tax year **2026**. The federal lowest-bracket rate is **14%** (down from 15% in 2025). YMPE is **$74,600**, YAMPE **$85,000**, EI maximum insurable earnings **$68,900** at **1.63%**.

1. What the CRA tab shows

The CRA tab is rendered by `components/dashboard/cra/cra-view.tsx`. It surfaces four things, in this order:

- **Next-due hero card** — the earliest unremitted month whose due-date is in the future, surfaced via `RemittanceService.getNextDue()`. Shows the total owed plus a four-cell breakdown (Federal tax / Provincial tax / CPP ×2 / EI ×2.4).
- **Upcoming remittances** — all other unremitted months. Sorted newest-first.
- **History** — months that have been marked remitted (`remittedAt` is set in `RemittanceStore`).
- **Year-end T4 section** — counts employees and links to the Employees tab for slip generation. Not part of the monthly remittance math.

The two state actions in the tab — `markRemitted(monthKey)` and `unmark(monthKey)` — only flip the boolean in the remittance store. They do not change any dollar amount; the totals always trace back to the underlying payroll runs.

2. Aggregation — `RemittanceService`

The service has two inputs and one output:

Input 1	runs: PayrollRun[] — every payroll run in local storage. Only runs whose <font name
Input 2	remittedMap: Record<string, string> — a map of monthK
Output	MonthlyRemittance[] — one row per calendar month that contains at least one finali

2.1 Bucketing rule

Each finalized run is bucketed by the **calendar month of its** `payDate`:

```
monthKey = run.payDate.slice(0, 7) // 'YYYY-MM'
```

This matches the CRA rule that a remittance covers the month in which the cheque **was dated**, not the pay-period it covers. A run with period Mar 30 → Apr 12 and payDate Apr 15 is part of the **April** remittance.

2.2 Per-line accumulation

Inside each month bucket, every line on every finalized run is summed into four totals. The exact code is:

```
for (const line of run.lines) {
  acc.federalTax += line.federalTax;
  acc.provincialTax += line.provincialTax;
  acc.cpp += line.cppEmployee + line.cppEmployer
                      + line.cpp2Employee + line.cpp2Employer;
  acc.ei += line.eiEmployee + line.eiEmployer;
}
```

Three things to notice in that block — they are the source of every doubling/multiplier shown in the CRA breakdown strip:

- **CPP is added twice.** Employee and employer pay equal CPP contributions (the employer mirrors the employee dollar-for-dollar at 5.95%), and the employer remits **both** halves to CRA. The `cppEmployer` field is literally set equal to `cppEmployee` in the engine (`engine.ts` line ~196), so for any line *cpp_employee + cpp_employer = 2 × cpp_employee*. Same logic for CPP2 (4% on the YMPE→YAMPE band).
- **EI uses the 1.4× employer multiplier.** The line's `eiEmployer` = `eiEmployee × 1.4`, so the remitted EI for a line equals *employee + employer = employee × 2.4*. This is why the hero strip is labelled "EI (×2.4)".
- **Income tax is added once.** Both federal and provincial income tax are pure employee-side deductions — the employer simply withholds them and forwards. There is no employer-side tax to add.

2.3 Rounding

Each per-line field on a finalized run has already been rounded to two decimals by the engine. The service sums them at full precision and applies one more `round2()` on each of the four monthly subtotals AND on the grand total, so the displayed `federal + provincial + cpp + ei` always equals `total` without a stray penny.

2.4 Due date

Due date is the **15th of the month following** the monthKey, computed deterministically without any timezone math:

```
function dueDateFor(monthKey: string): string {
  const [y, m] = monthKey.split('-').map(Number);
  const nextMonth = m === 12 ? 1 : m + 1;
  const nextYear = m === 12 ? y + 1 : y;
  return `${nextYear}-${pad(nextMonth)}-15`;
}
```

Scope note. The 15th-of-next-month rule is for *Threshold-1 / regular remitters* — the default for most small Canadian employers. Accelerated remitters (Threshold 2, AMWA $25k–$99,999.99) and quarterly remitters (AMWA < $1,000 + good compliance) are out of scope for this prototype. The `RemittanceService` always uses the Threshold-1 rule.

3. Federal income tax (per line)

Source: `lib/payroll/federal-tax.ts` + the BPA helper in `constants.ts`.

3.1 Algorithm

```
calculateFederalTax({ periodTaxableIncome, payPeriods }):
  annualized = periodTaxableIncome × payPeriods
  annual_gross_tax = sum(bracket × rate)
  bpa_credit = federalBPA(annualized) × 0.14
  annual_net = max(0, annual_gross_tax − bpa_credit)
  period_federal = annual_net / payPeriods
```

This mirrors the CRA T4127 "Option 1" method: annualize the period's taxable income, run it through the bracket table, subtract the BPA credit (always at the lowest bracket rate), then divide back down to a per-period amount.

3.2 The 2026 bracket table

Annualized taxable income	Marginal rate
$0 – $58,523	14%
$58,524 – $117,045	20.5%
$117,046 – $181,440	26%
$181,441 – $258,482	29%
Over $258,482	33%

The first bracket dropped from 15% to 14% for 2026 under the federal Tax Cut for the Middle Class (full-year rate). Every paystub, the BPA credit, and the K2 (CPP+EI) credit are sensitive to this change.

3.3 BPA with phase-out

The *federalBPA* helper returns a piecewise-linear function of annual income, then the engine multiplies that by the lowest bracket rate (14%) to get the dollar credit:

Annual income band	Effective BPA
≤ $181,440	$16,452 (enhanced full BPA)
$181,441 – $258,481	linear $16,452 → $14,829
≥ $258,482	$14,829 (old indexed BPA)

```
Linear interpolation formula used in code:
  t = (income − 181440) / (258482 − 181440)
  BPA = 16452 − (16452 − 14829) × t
```

3.4 K2 credit (CPP + EI)

After computing the per-period federal tax above, the engine applies the K2 credit in `engine.ts`:

```
cppEiCredit = (cppEmployee + cpp2Employee + eiEmployee) × 0.14
federalTax = max(0, federalTax − cppEiCredit)
```

This is what CRA calls the K2 factor — the employee gets the lowest-bracket rate as a non-refundable credit against the period's CPP + CPP2 + EI deductions. It is what makes the federal withholding feel much smaller than a naive "income × bracket rate" calculation.

4. Provincial income tax (per line)

Source: `lib/payroll/provincial-tax.ts` + the per-province table in `constants.ts`.

4.1 Algorithm

```
calculateProvincialTax({ province, periodTaxableIncome, payPeriods }):
  cfg = PROVINCIAL_TAX[province]
  annualized = periodTaxableIncome × payPeriods
  gross = sum(bracket × rate) // same taxOnIncome helper
  bpaCredit = cfg.basicPersonalAmount × cfg.brackets[0].rate
  tax = max(0, gross − bpaCredit)
  if (cfg.surtax) tax += sum_of_surtax_tiers(tax)
  return tax / payPeriods
```

Same shape as federal — annualize, bracket-tax, BPA credit at the **lowest provincial rate**, divide back down. The one twist is Ontario's surtax, which is computed *on top of* the base provincial tax (not on income directly).

4.2 2026 provincial constants (NorthPay supports 9 provinces)

Province	Top bracket / rate	BPA	Notes
ON	$220k+ @ 13.16%	$12,989	+ surtax tiers
BC	$265k+ @ 20.5%	$13,216	lowest 5.60% (Feb 2026 budget)
AB	$370k+ @ 15%	$22,769	NEW 8% bottom bracket
MB	$101k+ @ 17.4%	$15,780	indexation paused for 2026
SK	$156k+ @ 14.5%	$20,381	+$500 Affordability Act
NS	$157k+ @ 21%	$11,932	BPA now max-for-all
NB	$194k+ @ 19.5%	$13,664	2% indexation
PE	$81k+ @ 16.7%	$14,525	3-bracket structure
NL	$1.14m+ @ 21.8%	$15,000	BPA jumped to $15k

Quebec is intentionally *not* supported — Quebec has its own revenue agency (Revenu Québec, RQ) and runs separate QPP/QPIP withholding rules. NorthPay treats QC as "Coming soon".

4.3 Ontario surtax (the one province with tiers on top)

After Ontario's base provincial tax is computed, two additional tiers are layered on:

Base ON tax exceeds…	Surtax rate on the excess
$5,818	20%
$7,446	36%

Both tiers are additive — if base ON tax is above $7,446, the employee pays 20% surtax on the band $5,818→$7,446 AND 36% surtax on everything above $7,446.

5. CPP and CPP2 (per line)

Source: `lib/payroll/cpp.ts`.

5.1 The 2026 CPP constants

YBE (Year's Basic Exemption)	$3,500
YMPE (Year's Max Pensionable Earnings)	$74,600
YAMPE (Year's Additional Max Pensionable Earnings)	$85,000
Employee rate (and employer mirror)	5.95%
CPP2 rate (YMPE → YAMPE band)	4.00%
Max CPP1 contribution (employee or employer)	($74,600 − $3,500) × 5.95% = $4,230.45
Max CPP2 contribution (employee or employer)	($85,000 − $74,600) × 4% = $416.00

5.2 CPP1 formula

```
calculateCPP({ pensionableEarnings, payPeriods, ytdCpp }):
  periodExemption = 3500 / payPeriods
  taxableForCPP = max(0, pensionableEarnings − periodExemption)
  contribution = taxableForCPP × 0.0595
  remaining = 4230.45 − ytdCpp
  return min(contribution, remaining)
```

The YBE is distributed evenly across pay periods — bi-weekly means $3,500 / 26 = $134.62 of earnings each cheque are exempt from CPP. The capping step is what NorthPay calls the **YTD-aware cap**: it prevents the common bug where each period is computed standalone and the employee silently over-contributes once they pass the annual maximum.

5.3 CPP2 formula (band overlap method)

```
calculateCPP2({ ytdPensionableEarnings, periodPensionableEarnings, ytdCpp2 }):
  yearStart = ytdPensionableEarnings
  yearEnd = yearStart + periodPensionableEarnings
  overlap = max(0, min(yearEnd, 85000) − max(yearStart, 74600))
  contribution = overlap × 0.04
  return min(contribution, 416 − ytdCpp2)
```

CPP2 is only owed on the slice of earnings that fell **inside the YMPE→YAMPE band ($74,600 → $85,000)** during this pay period. The function computes the geometric overlap of [yearStart, yearEnd] with [74,600, 85,000], multiplies by 4%, and caps at $416.00. This produces zero CPP2 for low earners, partial CPP2 for mid earners who cross YMPE mid-year, and full $416 for high earners.

5.4 Employer side

There is no separate "calculate employer CPP" function. The engine sets `cppEmployer = cppEmployee` and `cpp2Employer = cpp2Employee` directly (`engine.ts` lines ~196–197) — CPP is a matched-contribution program. Both halves are remitted by the employer to CRA, which is why the CRA tab adds them together.

6. EI (per line)

Source: `lib/payroll/ei.ts`.

6.1 The 2026 EI constants

MIE (Max Insurable Earnings)	$68,900
Employee rate	1.63% (down from 1.64% in 2025)
Employer multiplier	1.4 × employee premium
Annual max employee premium	$68,900 × 1.63% = $1,123.07
Annual max employer premium	$1,123.07 × 1.4 = $1,572.30

6.2 Employee EI formula

```
calculateEI({ insurableEarnings, ytdEi }):
  premium = insurableEarnings × 0.0163
  remaining = 1123.07 − ytdEi
  return min(premium, remaining)
```

Note that, unlike CPP, EI has **no basic exemption** — the first dollar of insurable earnings is subject to the 1.63% premium. The YTD cap is the only thing that limits the employee's total annual contribution to $1,123.07.

6.3 Employer EI formula

```
calculateEIEmployer(employeePremium):
  return employeePremium × 1.4
```

The 1.4× multiplier is statutory. Combined with the employee side, the employer remits **2.4 ×** each EI dollar deducted from the employee — which is what the CRA tab's hero strip labels as "EI (×2.4)".

7. A worked example: one month, one employee

Let's walk a single bi-weekly pay run through the full pipeline and show how it lands in the CRA tab. Assumptions:

- Employee: Ontario resident, salaried at **$78,000/yr**, bi-weekly pay frequency (26 periods).
- Pay period: April 14 → April 27, 2026. payDate: April 28, 2026.
- No bonus, no stat pay, vacation in "accrue" mode (so it doesn't add to gross this period).
- This is the employee's first run of the year — YTD CPP, YTD EI, YTD pensionable earnings all start at $0.

7.1 Gross pay

```
Annual salary divided by periods:
  78,000 / 26 = $3,000.00
```

7.2 CPP1

```
periodExemption = 3,500 / 26 = 134.6154
taxableForCPP = 3,000 − 134.6154 = 2,865.3846
cppEmployee = 2,865.3846 × 0.0595 = $170.49
(YTD cap of $4,230.45 not yet reached → no clip)
```

7.3 CPP2

```
yearStart = 0, yearEnd = 3,000
overlap = max(0, min(3000, 85000) − max(0, 74600)) = 0
cpp2Employee = $0.00 (haven't crossed YMPE yet)
```

7.4 EI

```
eiEmployee = 3,000 × 0.0163 = $48.90
eiEmployer = 48.90 × 1.4 = $68.46
```

7.5 Federal tax

```
annualized = 3,000 × 26 = 78,000
bracket 1 (0 → 58,523) × 14% = 8,193.22
bracket 2 (58,524 → 78,000) × 20.5% = 3,992.59
annual_gross_tax = 12,185.81
BPA = 16,452 (income below phase-out)
bpa_credit = 16,452 × 14% = 2,303.28
annual_net = 12,185.81 − 2,303.28 = 9,882.53
period_federal = 9,882.53 / 26 = 380.10
```

```
Then the K2 (CPP + EI) credit is applied:
  cppEiCredit = (170.49 + 0 + 48.90) × 14% = 30.71
  federalTax = 380.10 − 30.71 = $349.39
```

7.6 Provincial tax (Ontario)

```
annualized = 78,000
bracket 1 (0 → 53,891) × 5.05% = 2,721.49
bracket 2 (53,892 → 78,000) × 9.15% = 2,205.88
annual_gross_tax = 4,927.37
```

```
ON BPA = 12,989; bpa_credit = 12,989 × 5.05% = 655.94
annual_net = 4,927.37 − 655.94 = 4,271.43
Surtax: base 4,271.43 < 5,818 ⇒ no surtax
period_provincial = 4,271.43 / 26 = $164.29
```

7.7 The PD7A contribution from this one line

On the CRA tab, this single line contributes the following to April 2026's remittance bucket:

Component	Amount	How it's added
Federal tax	$349.39	+ line.federalTax (×1)
Provincial tax	$164.29	+ line.provincialTax (×1)
CPP1 employee	$170.49	+ line.cppEmployee
CPP1 employer	$170.49	+ line.cppEmployer (= employee)
CPP2 employee	$0.00	+ line.cpp2Employee
CPP2 employer	$0.00	+ line.cpp2Employer
EI employee	$48.90	+ line.eiEmployee
EI employer	$68.46	+ line.eiEmployer (= 1.4 × emp)

Folded into the four CRA-tab buckets:

Bucket on the CRA tab	This line's contribution
Federal tax	$349.39
Provincial tax	$164.29
CPP (×2)	$340.98 (170.49 × 2)
EI (×2.4)	$117.36 (48.90 + 68.46)
Total	$972.02

8. End-to-end data flow

Putting all of the above together, here is the full path a dollar takes from the moment the operator clicks *Finalize payroll* to the moment it shows up on the CRA tab:

#	Step	Lives in
1	LifecycleService transitions status preview → finalized and appends to the payroll repository.	lib/payroll/lifecycle.ts
2	loyee / cpp2Employer / eiEmployee / eiEmployer fields, using YTD-aware CPP and EI caps.	lib/payroll/engine.ts
3	store re-hydrates from localStorage and CRAView reads usePayrollRuns + useRemittance…	components/dashboard/cra/cra-view.tsx
4	.slice(0,7), and accumulates federal / provincial / (cpp×2 + cpp2×2) / (ei + ei×1.4) per month.	lib/services/remittance.ts
5	ed a dueDate (15th of next month) and a remitted boolean (from the operator's prior actions).	RemittanceService.getMonthly
6	nremitted month whose due-date is in the future and total > 0 — this becomes the hero card.	RemittanceService.getNextDue
7	into the remitted map; the service re-runs and that month moves from Upcoming → History.	lib/store/remittance.ts

9. What is intentionally *not* computed

A few notes about scope, so it's clear what the CRA tab does and does not produce:

- **Quarterly / accelerated remitters.** The 15th-of-next-month due date is hard-coded. CRA's actual remitter thresholds — quarterly (AMWA < $1,000), regular, Threshold 1 (AMWA $25k–$99,999.99), Threshold 2 (AMWA ≥ $100,000) — would each require their own due-date logic. Out of scope.
- **Quebec.** QC employees aren't supported at all; the engine doesn't accept QC as a province code, so the CRA tab will never see Quebec-side QPP / QPIP / Revenu Québec remittances.
- **WSIB / WCB.** Workers' compensation premiums are a *provincial* remittance, not a CRA one, and are not part of the PD7A. Not surfaced anywhere in the app.
- **RPP/RRSP contributions.** If an employee has an employer-sponsored registered pension plan, that affects T4 box 20 and box 52 but does not flow into the PD7A line items. Out of scope.
- **T4 summary reconciliation.** The Year-end · 2026 section links to Employees → T4 PDF generation. It does not yet reconcile the sum of monthly PD7A remittances against the T4 summary that would be filed at year-end. That's a separate verification step.

10. File index — where each calculation lives

Concern	File
CRA tab UI	components/dashboard/cra/cra-view.tsx
Monthly aggregation	lib/services/remittance.ts
Operator status toggles	lib/store/remittance.ts
Payroll engine (per-line math)	lib/payroll/engine.ts
Federal tax (brackets, BPA)	lib/payroll/federal-tax.ts

Concern	File
Provincial tax (brackets, surtax, BPA)	lib/payroll/provincial-tax.ts
CPP1 + CPP2	lib/payroll/cpp.ts
EI	lib/payroll/ei.ts
All 2026 constants	lib/payroll/constants.ts
YTD accumulator	lib/services/ytd.ts
Run lifecycle (preview → finalized → voided)	lib/services/lifecycle.ts (consumes engine)

Document generated from NorthPay source as of the current working tree. All constants reflect tax year 2026 as defined in lib/payroll/constants.ts.